BrowserBox Customer Guide

DOSAYGO

Current as of BrowserBox v16.2.0
April 14, 2026

Status: Provisional customer reference. Supported commands, environment variables, login-link parameters, and operational patterns are subject to change in future BrowserBox releases.

Contents

1 Purpose **4**

2 Installation and Customer Resources **4**
2.1 Official install entrypoints . . . 4
2.2 Suggested first run . . . 4
2.3 Support and public references . . . 4

3 Fast Start **5**
3.1 Standard local workflow . . . 5
3.2 Core commands . . . 5

4 Customer Customization Surfaces **5**
4.1 Login-link URL parameters . . . 5
4.2 Homepage selection . . . 6
4.3 Custom cursor . . . 6
4.4 UI theme override . . . 6
4.5 Idle shutdown after disconnect . . . 6

5 Customer Policy Surface **6**
5.1 What policy controls . . . 6
5.2 Canonical policy fields . . . 7
5.3 Allowed values and defaults . . . 7
5.3.1 Binary feature controls . . . 7
5.3.2 Enumerated policy fields . . . 8
5.3.3 List-valued policy fields . . . 8
5.3.4 Where customers can verify allowed values . . . 8
5.4 Sensitive actions BrowserBox can allow or deny . . . 9
5.5 Navigation and private-network policy . . . 9
5.5.1 Navigation modes . . . 9
5.5.2 Private-network controls . . . 10
5.6 Certificate-error policy . . . 10

5.7 Browser chrome policy modes . . . 10
5.8 Server policy versus embedder policy . . . 11
5.9 Customer policy CLI . . . 11
5.9.1 Useful commands . . . 11
5.10 Operational visibility . . . 12

6 Embedding and Origin Allowlisting **12**
6.1 Server-side embedding allowlist . . . 12
6.2 Canonical embedder controls . . . 13

7 Setup and Backend Modes **13**
7.1 `bbx setup` . . . 13
7.2 Reverse proxy and HTTP backend . . . 13
7.3 External TLS hint . . . 14

8 Tunnel and Network-Oriented Commands **14**
8.1 `bbx ng-run` . . . 14
8.2 `bbx tor-run` . . . 14
8.3 `bbx zt-run` . . . 14
8.4 `bbx cf-run` . . . 14

9 Zeta Mode and `hosts.env` **14**

10 TLS and Certificates **15**
10.1 BrowserBox-managed TLS . . . 15
10.2 Customer-managed certificates . . . 15

11 Customer-Relevant Environment Variables **15**
11.1 Core customer-facing variables . . . 15
11.2 Examples . . . 16

12 Licensing and Occupancy Visibility **17**
12.1 Available today . . . 17
12.2 What `bbx vacancy` means . . . 17
12.3 What is not surfaced here . . . 17

13 Practical Recipes **17**
13.1 Embedded BrowserBox without the standard UI . . . 17
13.2 BrowserBox behind a reverse proxy . . . 18
13.3 Check whether a seat is currently available . . . 18

14 Transport and Streaming **18**
14.1 WebRTC (default) . . . 18
14.2 WebSocket fallback . . . 18
14.3 Streaming diagnostics . . . 18

15 Flipbook Recording **18**
15.1 Quick start . . . 19
15.2 How it works . . . 19
15.3 Output structure . . . 19
15.4 Frame normalization . . . 19
15.5 Cloudflare Pages deployment . . . 20
15.6 Relevant environment variables . . . 20
15.7 Direct site generation . . . 20

16 Future Directions **20**

1 Purpose

This document combines the customer-facing BrowserBox setup guide and the customer-relevant configuration reference into one printable manual. It is intended to answer the questions customers usually ask first:

- How do I install BrowserBox on macOS, Linux, or Windows?
- How do I set BrowserBox up and start it?
- What can I customize?
- How do reverse proxy, HTTP mode, and certificates work?
- How do I embed BrowserBox safely in my own application?
- What do the tunnel-oriented commands mean?
- What licensing visibility exists today?

2 Installation and Customer Resources

2.1 Official install entrypoints

BrowserBox installation guidance is published at `https://browserbox.io`. The current public install entrypoints are:

- macOS and Linux: `curl -fsSL https://browserbox.io/install.sh | bash`
- Windows: `irm https://browserbox.io/install.ps1 | iex`

The Windows installer exists and is useful for getting a BrowserBox binary onto a Windows machine, but the Windows PowerShell `bbx` surface is not command-parity compatible with the bash-oriented `bbx` tool documented in this guide. For that reason, this guide documents the bash CLI behavior and uses the Windows path only as an installation entrypoint.

2.2 Suggested first run

After installation, the normal customer workflow is:

```
bbx certify YOUR_KEY
bbx setup -p 8888
bbx start
```

If you are evaluating BrowserBox, obtain your license through `https://browserbox.io`. Commercial licensing and evaluation paths are surfaced there, and the site links through to the current purchase flow.

2.3 Support and public references

- Product site and license purchasing: `https://browserbox.io`
- Public GitHub repository: `https://github.com/BrowserBox/BrowserBox`
- Support email: support@dosaygo.com

3 Fast Start

3.1 Standard local workflow

```
bbx stop
bbx setup -p 8888
bbx start
```

When you run `bbx setup -p 8888`, port `8888` becomes the main BrowserBox service port. Related service ports are derived from the main port:

- audio: main port - 2
- docs: main port - 1
- devtools: main port + 1

BrowserBox writes the current login link to:

```
~/.config/dosaygo/bbpro/login.link
```

3.2 Core commands

Command	Purpose
`bbx setup`	Configure port, hostname, token, zeta mode, and backend mode.
`bbx start`	Start BrowserBox for the current user.
`bbx stop`	Stop BrowserBox for the current user.
`bbx status`	Check whether BrowserBox appears reachable and running.
`bbx logs`	View BrowserBox logs.
`bbx certify`	Validate the current license and reserve a seat when applicable.
`bbx vacancy`	Show total, occupied, vacant, leased, and reserved seat counts, plus any local reservation metadata on the machine.
`bbx ng-run`	Run the nginx-oriented workflow.
`bbx tor-run`	Run BrowserBox with Tor integration.
`bbx zt-run`	Run BrowserBox on a ZeroTier network.
`bbx cf-run`	Run BrowserBox through a Cloudflare tunnel.

4 Customer Customization Surfaces

4.1 Login-link URL parameters

BrowserBox generates login links with query parameters that affect customer-visible behavior.

Parameter	Meaning
`ui=false`	Hides the standard BrowserBox chrome for embedding or kiosk-style use.
`url=`*`https-url`*	Opens a target page on first load.
`mid=`*`machine-id`*	Infrastructure routing affinity in multi-machine deployments. This is not a user identity or a seat identifier.
`uiTheme=`*`theme`*	Requests a UI theme when the server accepts that theme value.

When `ui=` is explicitly present, BrowserBox treats that as embedder-owned configuration and locks the user toggle accordingly.

4.2 Homepage selection

Use either of these variables:

```
export BBX_HOME_PAGE="https://intranet.example.com"
# or
export BBX_DEFAULT_HOME_PAGE="https://intranet.example.com"
```

The value must be a valid `http:` or `https:` URL. Invalid values fall back to https://duckduckgo.com.

4.3 Custom cursor

```
export BBX_CUSTOM_CURSOR="/absolute/path/to/cursor.png"
bbx start
```

BrowserBox reads the local file at startup and uses it as the remote cursor image.

4.4 UI theme override

```
export BBX_UI_THEME_OVERRIDE=dark
bbx start
```

This sets the default BrowserBox UI theme for the login flow. A login-link `uiTheme=` parameter can also request a theme when accepted by the server.

4.5 Idle shutdown after disconnect

```
export BBX_SHUTDOWN_IDLE_MS=7200000
bbx start
```

This controls how long BrowserBox waits after all clients disconnect before shutting down.

If you run with `BBX_MINIMAL_MODE="true"` and do not set `BBX_SHUTDOWN_IDLE_MS`, BrowserBox uses `BBX_MINIMAL_MODE_IDLE_MS` as the fallback idle shutdown duration.

5 Customer Policy Surface

5.1 What policy controls

BrowserBox policy is the customer-facing runtime control surface that answers two distinct questions:

- Which sensitive actions are allowed or denied?
- Which browsing targets may the remote browser reach?

The canonical persisted policy file lives at:

```
~/.config/dosaygo/bbpro/policy/policy.json
```

BrowserBox normalizes and validates that file before applying it. If a field is omitted, BrowserBox fills in safe defaults for the current schema.

The allowed values listed in this section are based on BrowserBox's current runtime validation and normalization surface, not on a separate aspirational specification. In practice, customers should be able to trust this section because these values are the same ones BrowserBox validates at load time.

5.2 Canonical policy fields

The customer-relevant policy bundle currently includes these high-level surfaces:

Field	Meaning
`transport.certificateErrors`	Whether BrowserBox enforces certificate validation or explicitly allows self-signed / invalid certificates.
`navigation.mode`	Top-level navigation mode: `open`, `allowlist`, or `blocklist`.
`navigation.allowlist`	Allowed navigation targets when `navigation.mode=allowlist`. Supports exact host, exact origin, exact URL, and wildcard host patterns.
`navigation.blocklist`	Denied navigation targets when `navigation.mode=blocklist`.
`navigation.privateNetwork.mode`	Whether private/local targets are denied by default or controlled through an allowlist.
`navigation.privateNetwork.allowlist`	Explicit private/local destinations that may be reached, including exact hosts, IPs, and IPv4 CIDRs.
`featureControls.copy`	Controls copy actions, including modifier shortcuts and context-menu copy paths.
`featureControls.paste`	Controls paste actions, including text insertion into the remote browser.
`featureControls.downloads`	Controls download routes and archive-serving surfaces.
`featureControls.uploads`	Controls upload routes and file input surfaces.
`featureControls.devtools`	Controls DevTools access.
`featureControls.audio`	Controls the audio sidecar service.
`featureControls.automation`	Controls the BrowserBox automation surface used by the webview API.
`featureControls.contextMenu`	Controls whether the remote-page context menu may open.
`featureControls.browserChrome`	Controls BrowserBox's own visible browser chrome mode rather than allowing or denying an action.

5.3 Allowed values and defaults

The most important values customers need to know are listed here directly.

5.3.1 Binary feature controls

Unless stated otherwise, feature controls are binary decisions:

Field	Allowed values	Default / note

`featureControls.copy`	`allow`, `deny`	Required in the persisted policy bundle.
`featureControls.paste`	`allow`, `deny`	Required in the persisted policy bundle.
`featureControls.downloads`	`allow`, `deny`	Required in the persisted policy bundle.
`featureControls.uploads`	`allow`, `deny`	Required in the persisted policy bundle.
`featureControls.devtools`	`allow`, `deny`	Required in the persisted policy bundle.
`featureControls.audio`	`allow`, `deny`	Required in the persisted policy bundle.
`featureControls.automation`	`allow`, `deny`	Required in the persisted policy bundle.
`featureControls.contextMenu`	`allow`, `deny`	If omitted, BrowserBox currently defaults this field to `allow` for backward compatibility.

5.3.2 Enumerated policy fields

Field	Allowed values	Default / note
`transport.certificateErrors`	`enforce`, `ignore-all`	Defaults to `enforce`.
`navigation.mode`	`open`, `allowlist`, `blocklist`	Required in the persisted policy bundle.
`navigation.privateNetwork.mode`	`deny`, `allowlist`	Defaults to `deny`.
`featureControls.browserChrome`	`toggle`, `api`, `always-show`, `always-hide`	Defaults to `toggle`.

5.3.3 List-valued policy fields

Field	Allowed entry form
`navigation.allowlist`	Array of non-empty strings. Entries may be exact hosts, exact origins, exact URLs, or wildcard host patterns.
`navigation.blocklist`	Array of non-empty strings. Entries follow the same matching forms as the navigation allowlist.
`navigation.privateNetwork.allowlist`	Array of non-empty strings. Entries may be exact hosts, exact IPs, or IPv4 CIDRs.

5.3.4 Where customers can verify allowed values

Customers do not have to guess. BrowserBox exposes the current policy surface through the policy CLI:

- `browserbox policy get` prints the persisted policy bundle.

- `browserbox policy resolve` prints the normalized effective policy that BrowserBox is actually using.
- `browserbox policy validate -file policy.json` validates a candidate policy file against the current code and reports invalid enum values directly.

For example, if a customer uses an unsupported mode, BrowserBox validation reports the current accepted set, such as:

```
featureControls.browserChrome must be one of:
toggle, api, always-show, always-hide
```

5.4 Sensitive actions BrowserBox can allow or deny

These actions currently route through BrowserBox's policy authorization layer:

Action	Customer-visible effect
navigate	Page navigation, history navigation, new-tab target creation, and clean-slate navigation flows.
uploads	File upload routes and file chooser / file input flows.
downloads	Download and archive-serving routes.
devtools	BrowserBox DevTools service and its websocket bridge.
audio	BrowserBox audio service.
copy	Keyboard and context-menu copy paths.
paste	Keyboard and API-driven paste / insert-text paths.
automation	The automation methods exposed through the webview API, such as selector waits, clicks, typing, and evaluation.
contextmenu	The remote-page context menu UI.

All of these controls are binary `allow` / `deny` decisions except `featureControls.browserChrome`, which uses explicit UI visibility modes described below.

5.5 Navigation and private-network policy

BrowserBox separates public browsing control from private/local reachability:

- `navigation.allowlist` answers "where may the RBI tab browse?"
- `navigation.privateNetwork.allowlist` answers "which private or local destinations may any request reach?"
- `ALLOWED_EMBEDDING_ORIGINS` is separate and only controls which parent pages may embed BrowserBox

5.5.1 Navigation modes

Mode	Behavior
open	Any browsing target is allowed, subject to the separate private-network boundary.
allowlist	Only configured allowlist entries may be reached.

blocklist	All targets are allowed except those matching the configured blocklist.

Allowlist and blocklist entries support:

- exact host names, for example `example.com`
- exact origins, for example `https://secure.example.com`
- exact URLs, for example `https://secure.example.com/tools?mode=estimate`
- wildcard host patterns, for example `*.customer.example`

5.5.2 Private-network controls

Private-network policy defaults to deny. Customer policy may widen that boundary explicitly:

Field	Meaning
mode=deny	Denies private and local targets such as RFC1918 addresses and localhost by default.
mode=allowlist	Keeps the private/local boundary in place but permits explicit entries.
allowlist entries	Exact hosts, exact IPs, and IPv4 CIDRs, for example `10.0.0.5`, `10.0.0.0/8`, and `localhost`.

5.6 Certificate-error policy

Certificate handling is now policy-driven rather than hidden behind a permanent launch flag.

Mode	Behavior
enforce	Default. BrowserBox enforces certificate validation and will not silently accept certificate errors.
ignore-all	Explicit opt-in for environments that need to reach self-signed or otherwise invalid HTTPS targets.

5.7 Browser chrome policy modes

`featureControls.browserChrome` does not allow or deny a risky action. Instead, it governs whether BrowserBox's own visible browser chrome is shown, hidden, or user-toggleable.

Mode	Visible UI	User toggle	Meaning
toggle	visible by default	yes	Legacy-compatible mode. The user may show or hide BrowserBox chrome unless the embedder locks it.
api	visible by default	no	The embedder may drive UI visibility through the API, but the end user may not toggle it directly.
always-show	always visible	no	BrowserBox chrome must remain visible.

always-hide	always hidden	no	BrowserBox chrome stays hidden for kiosk-style or tightly embedded experiences.

If the embedder sets `ui-visible` or `allow-user-toggle-ui`, those values may further restrict the experience. They cannot widen what the server policy allows.

5.8 Server policy versus embedder policy

BrowserBox has two relevant policy layers:

1. **Server policy**, stored in `policy.json`, which is authoritative.
2. **Embedder policy hints**, supplied through the canonical webview surface, which may only restrict and never widen the effective policy.

- Server policy is the floor.
- Embedder restrictions may narrow capabilities further.
- An embedder cannot override a server-side deny into an allow.

5.9 Customer policy CLI

BrowserBox ships a policy CLI so customers can inspect, initialize, validate, and test the effective policy surface.

```
browserbox policy help
browserbox policy where
browserbox policy baselines
browserbox policy controls [--json]
browserbox policy get
browserbox policy resolve
browserbox policy check --action <name> [--url <https://...>] [--source <tag>]
browserbox policy trace [--last <n>]
browserbox policy init [--baseline regulated|compat]
browserbox policy reset [--baseline regulated|compat]
browserbox policy set --file <path/to/policy.json>
browserbox policy validate [--file <path/to/policy.json>]
```

5.9.1 Useful commands

Command	Meaning
browserbox policy where	Prints the canonical persisted policy path.
browserbox policy baselines	Lists supported baseline names.
browserbox policy controls –json	Prints the documented NIST 800-53 and FIPS 140 control surfaces.
browserbox policy get	Prints the currently persisted customer policy bundle.
browserbox policy resolve	Prints the compiled / normalized effective policy snapshot that BrowserBox is using.

browserbox policy check –action ...	Evaluates a single action decision. Exit code `0` means allow; exit code `2` means deny.
browserbox policy trace –last 20	Shows recent policy decisions from the decision trace log.
browserbox policy init –baseline compat	Creates the initial policy file from a named baseline.
browserbox policy reset –baseline regulated	Replaces the policy file with a named baseline.
browserbox policy set –file policy.json	Loads and validates a customer-supplied policy file, then persists it.
browserbox policy validate	Validates the current persisted policy or an explicitly supplied file.

5.10 Operational visibility

When policy blocks an action, BrowserBox emits both customer-visible and operator-visible signals:

- client-side policy-denied events
- server log entries prefixed with `[policy-decision]`
- a persistent trace file at `~/.config/dosaygo/bbpro/policy/decisions.ndjson`

6 Embedding and Origin Allowlisting

For the hosted BrowserBox SaaS service, the current REST API and embed reference is published at https://win9-5.com/api/. That reference covers both session-lifecycle REST endpoints and the canonical `<browserbox-webview>` custom element using the current method and event names.

6.1 Server-side embedding allowlist

For self-hosted BrowserBox embeds, the main server-side allowlist is:

```
export ALLOWED_EMBEDDING_ORIGINS="https://app.example.com https://*.customer.example:*"
```

This controls which parent pages may embed BrowserBox.
Supported patterns include:

- exact origins, for example `https://app.example.com`
- subdomain wildcards, for example `https://*.customer.example`
- trailing port wildcards, for example `https://app.example.com:*`
- both together, for example `https://*.customer.example:*`

Important boundary:

- `ALLOWED_EMBEDDING_ORIGINS` controls which parent pages may embed BrowserBox.
- It does not control where the remote browser may browse.
- Browsing targets are controlled by BrowserBox policy.

6.2 Canonical embedder controls

For new embeds, prefer explicit webview controls over encoding everything into the login URL:

- `embedder-origin="https://app.example.com"`
- `ui-visible="false"`
- `allow-user-toggle-ui="false"`

  Minimal example:

```
<browserbox-webview
  login-link="https://localhost:8888/login?token=..."
  embedder-origin="https://app.example.com"
  ui-visible="false"
  allow-user-toggle-ui="false"
></browserbox-webview>
```

7 Setup and Backend Modes

7.1 `bbx setup`

Customer-facing setup shape:

```
bbx setup [--port|-p <port>] [--hostname|-h <hostname>]
          [--token|-t <token>] [--zeta|-z]
          [--backend <http|https>]
          [--flipbook-record <dir>]
          [--flipbook-description <text>]
```

- `-port` chooses the main BrowserBox service port.
- `-hostname` sets the hostname customers use.
- `-token` sets the login token explicitly instead of generating one.
- `-zeta` enables host-per-service mode.
- `-backend http|https` selects whether BrowserBox itself serves HTTP or HTTPS.
- `-flipbook-record` enables flipbook recording (see Section 15).
- `-flipbook-description` sets an optional description for the recording.

  Runtime values are written to:

```
~/.config/dosaygo/bbpro/test.env
```

7.2 Reverse proxy and HTTP backend

If BrowserBox sits behind nginx, Caddy, or another edge proxy that terminates TLS:

```
bbx setup --backend http
```

The older compatibility form is still available:

```
bbx setup --http-only
```

7.3 External TLS hint

If BrowserBox is behind an edge that terminates HTTPS and BrowserBox itself is serving plain HTTP, the runtime can be told to treat the frontend as secure:

```
export BBX_EXTERNAL_TLS=true
bbx start
```

This is useful when a reverse proxy or cloud edge is doing TLS termination for customer traffic.

8 Tunnel and Network-Oriented Commands

8.1 `bbx ng-run`

```
bbx ng-run
```

This is the nginx-oriented workflow. It will trigger zeta-mode setup when needed, expects `hosts.env` support for per-service hostnames, runs nginx setup, and then starts BrowserBox.

8.2 `bbx tor-run`

```
bbx tor-run [--anonymize|--clearnet-only] [--onion|--no-onion]
```

This mode sets up Tor integration and can expose BrowserBox through onion routing.

8.3 `bbx zt-run`

```
bbx zt-run
```

This is the ZeroTier overlay-network entry point for private-network access.

8.4 `bbx cf-run`

```
bbx cf-run
```

This is the Cloudflare Tunnel entry point for public HTTPS exposure through Cloudflare's edge.

9 Zeta Mode and `hosts.env`

The `-zeta` option enables host-per-service mode. BrowserBox then expects:

```
~/.config/dosaygo/bbpro/hosts.env
```

At runtime, `bbx run` reads `ADDR_<PORT>` from that file to construct the login link for the main service port.

Use zeta mode when:

- nginx is doing hostname-based proxying
- you want separate hostnames per BrowserBox service
- you want clean standard-port edge routing

If `hosts.env` is missing, `bbx run` fails fast in zeta mode.

10 TLS and Certificates

10.1 BrowserBox-managed TLS

On normal local or self-hosted setups, `bbx setup` ensures host resolution is sane, obtains certificates through BrowserBox's TLS flow, and starts BrowserBox in HTTPS mode when the expected certificate files exist.

10.2 Customer-managed certificates

If you want BrowserBox to use your own certificate material instead of BrowserBox-managed certificates, set:

```
export BBX_SSLCERTS_DIR="/path/to/your/certs"
export BBX_SKIP_CERT_COPY=1
```

The certificate directory must contain:

- `fullchain.pem`
- `privkey.pem`

This is the customer-facing manual override path for deployments that already have externally managed certificates.

Another common TLS pattern is to run `bbx setup -backend http` and terminate TLS in nginx, Caddy, or another reverse proxy.

11 Customer-Relevant Environment Variables

11.1 Core customer-facing variables

Name	Default	Meaning
`ALLOWED_EMBEDDING_ORIGINS`	unset	Space-separated allowlist of parent origins permitted to embed BrowserBox.
BBX_HTTP_ONLY	unset	Forces HTTP-only mode and skips HTTPS certificate generation. Useful behind a reverse proxy.
BBX_EXTERNAL_TLS	unset	Treats the frontend as secure when TLS is terminated upstream.
BBX_SSLCERTS_DIR	unset	Directory containing customer-managed TLS files such as `fullchain.pem` and `privkey.pem`.
BBX_SKIP_CERT_COPY	unset	Prevents BrowserBox-managed certificates from overwriting certificates in the configured certificate directory.
BBX_SHUTDOWN_IDLE_MS	2700000	Main idle shutdown after all clients disconnect.

`BBX_MINIMAL_MODE_IDLE_MS`	120000	Minimal-mode idle shutdown fallback.
BBX_HOME_PAGE	unset	Preferred home page URL.
`BBX_DEFAULT_HOME_PAGE`	https://duckduckgo.com	Fallback home page URL.
BBX_CUSTOM_CURSOR	unset	Path to a local image file used as the remote cursor image.
BBX_HOSTNAME	system hostname or prompt	Hostname used for BrowserBox setup.
BBX_UI_THEME_OVERRIDE	unset	Default login UI theme when a supported theme is requested.
`BBX_SHOW_STREAMING_STATS_OVERLAY`	unset	Displays the live streaming stats overlay in the BrowserBox client UI.
BBX_MAX_CONNECTIONS	5	Per-session client connection cap.
BBX_MAX_TABS	15	Maximum concurrent tabs per BrowserBox session.
BBX_CLEAN_SLATE	unset	When `true`, clears the browser user-data directory at startup. Useful for ephemeral deployments that need a fresh profile every launch.
BBX_MINIMAL_MODE	unset	Runs BrowserBox with only the main service (no audio, docs, or devtools sidecars). Useful for constrained environments or single-purpose kiosks.
BBX_DISABLE_WEBRTC	unset	When `true`, disables WebRTC entirely and forces WebSocket-only transport.
LICENSE_KEY	unset	Your BrowserBox product key. Can be set in the environment, passed to `bbx certify`, or persisted in `test.env` after first certification.

11.2 Examples

```
export ALLOWED_EMBEDDING_ORIGINS="https://app.example.com https://localhost:*"
export BBX_HTTP_ONLY=1
export BBX_EXTERNAL_TLS=true
export BBX_SSLCERTS_DIR="/path/to/certs"
export BBX_SKIP_CERT_COPY=1
export BBX_HOME_PAGE="https://intranet.example.com"
export BBX_CUSTOM_CURSOR="/absolute/path/to/cursor.png"
export BBX_SHUTDOWN_IDLE_MS=7200000
export BBX_UI_THEME_OVERRIDE=dark
export BBX_SHOW_STREAMING_STATS_OVERLAY=true
export BBX_MAX_CONNECTIONS=8
export BBX_MAX_TABS=24
bbx setup
bbx start
```

12 Licensing and Occupancy Visibility

12.1 Available today

- `bbx certify` validates the license and reserves a seat when possible.
- `bbx vacancy` queries the license server and reports total seats plus current occupied and vacant counts when the server supports occupancy summaries.
- Local reservation metadata is stored under `~/.config/dosaygo/bbpro/tickets/`.

12.2 What `bbx vacancy` means

```
bbx vacancy
```

For modern servers, `bbx vacancy` reports a customer-safe summary:

- total seats
- occupied seats now
- vacant seats now
- leased active
- reserved active

By default it does not expose seat IDs or deeper lease inventory to ordinary customer keys.

12.3 What is not surfaced here

This BrowserBox runtime still does not surface richer customer history such as:

- detailed current lease inventory for ordinary customer keys
- free-seat counts over time
- occupancy history

13 Practical Recipes

13.1 Embedded BrowserBox without the standard UI

```
export ALLOWED_EMBEDDING_ORIGINS="https://app.example.com"
export BBX_HOME_PAGE="https://app.example.com"
bbx setup -p 8888
bbx start
```

Then use a login link shaped like:

```
https://localhost:8888/login?token=...&ui=false
```

13.2 BrowserBox behind a reverse proxy

```
bbx stop
bbx setup --backend http -p 8080 --hostname browserbox.internal
bbx start
```

Then let nginx, Caddy, or a cloud load balancer terminate TLS and forward plain HTTP to BrowserBox.

13.3 Check whether a seat is currently available

```
bbx vacancy
```

This command reports the current seat summary, including total seats, occupied seats, vacant seats, active leases, and active reservations.

Some deeper occupancy reporting may vary by deployment version.

14 Transport and Streaming

BrowserBox uses an adaptive, ACK-paced streaming pipeline to deliver remote browser frames to connected clients. Two transport modes are available:

14.1 WebRTC (default)

By default, BrowserBox negotiates a peer-to-peer data channel between server and client. This delivers the lowest latency and highest frame throughput. The streaming pipeline observes round-trip time and automatically adjusts frame pacing.

14.2 WebSocket fallback

If WebRTC negotiation fails (firewalled UDP, restricted NAT, or `BBX_DISABLE_WEBRTC=true`), BrowserBox falls back to WebSocket binary transport. This works through any HTTP proxy or load balancer and still delivers real-time frames, though with slightly higher latency than WebRTC.

14.3 Streaming diagnostics

BrowserBox can show a live streaming stats overlay in the client UI with FPS, transport mode, round-trip time, and frame-flow status.

To display it:

```
export BBX_SHOW_STREAMING_STATS_OVERLAY=true
bbx start
```

This is useful for customer demos, troubleshooting, and performance checks when you want a visible real-time summary without opening developer tools.

15 Flipbook Recording

BrowserBox can record a browsing session as a *flipbook*—a self-contained static site of sequential JPEG frames with a JavaScript viewer. Flipbook recordings are produced directly from the internal screencast pipeline with negligible overhead: when recording is off, the only cost is a single boolean check per frame.

15.1 Quick start

```
bbx setup --flipbook-record ~/my-recording \
          --flipbook-description "Onboarding walkthrough"
bbx run
# ... use BrowserBox normally ...
bbx stop
```

When `bbx stop` is called, BrowserBox compiles the captured frames into a complete flipbook static site and (if Cloudflare `wrangler` is available) offers to deploy it to Cloudflare Pages.

15.2 How it works

1. `bbx setup` writes `BBX_FLIPBOOK_DIR` and optionally `BBX_FLIPBOOK_DESCRIPTION` to `~/.config/dosaygo/bbpro/test.env`.
2. At runtime, each screencast frame is written as a JPEG + JSON metadata pair to a temporary directory inside the BrowserBox config directory (`~/.config/dosaygo/bbpro/flipbook_recording/`).
3. On `bbx stop`, the built-in `browserbox flipbook-generate` command compiles the raw frames into a flipbook static site under the directory provided by `-flipbook-record`.

When recording is disabled (no `-flipbook-record` flag), **there is zero overhead** on the frame pipeline—only a single falsy-value check is evaluated per frame.

15.3 Output structure

Each recording produces a timestamped directory inside the flipbook base directory:

```
~/my-recording/
  2026-04-14T02-25-00-000Z--2026-04-14T02-30-00-000Z/
    site/
      index.html          # self-contained viewer
      manifest.json       # flipbook v1 manifest
      pages/
        000000.jpg        # contiguous, zero-padded frames
        000001.jpg
        ...
      assets/
        viewer.css
        viewer.js
        sw.js             # service worker for offline caching
      meta/
        provenance.json   # full per-frame metadata + recording info
```

Multiple runs with the same `-flipbook-record` directory produce separate timestamped subdirectories, each a self-contained flipbook site.

15.4 Frame normalization

Frame IDs from the internal pipeline may contain gaps (due to cast restarts or tab switches). The site generator performs a normalization pass: frames are sorted, paired by basename (JPEG with its JSON metadata), and re-indexed as a contiguous zero-padded sequence (`000000`, `000001`, ...). The original frame IDs are preserved in `meta/provenance.json` for traceability.

15.5 Cloudflare Pages deployment

If `wrangler` (the Cloudflare CLI) is available or can be installed, `bbx stop` will offer to deploy the generated flipbook site to Cloudflare Pages:

```
# Install wrangler if not already present
npm install -g wrangler

# Authenticate (one-time)
wrangler login

# Deployment happens automatically on bbx stop
```

The Pages project name is derived from the flipbook directory basename. If `wrangler` is not available, the site is still generated locally and can be served with any static file server.

15.6 Relevant environment variables

`BBX_FLIPBOOK_DIR`	Absolute path to the flipbook output directory. Set by `-flipbook-record`. When empty, recording is disabled.
`BBX_FLIPBOOK_DESCRIPTION`	Optional human-readable description embedded in the manifest and provenance metadata. Set by `-flipbook-description`.

15.7 Direct site generation

The flipbook site generator is available as a standalone command on the BrowserBox binary:

```
browserbox flipbook-generate <flipbookDir>
```

This reads raw frames from the config directory, compiles the flipbook site into a timestamped subdirectory of `<flipbookDir>`, and cleans up the temporary recording data. This is called automatically by `bbx stop` but can also be invoked manually.

16 Future Directions

Likely next customer-facing improvements:

- a published `hosts.env` schema with nginx examples
- a reverse-proxy cookbook for `-backend http`
- a separate customer-safe reference for supported login-link parameters